When Specifications Become Executable: How Spec-Driven Development Reshapes the Way We Build Software

This article was written with AI assistance. The build, observations, and editorial judgement are my own.

For most of the last decade, specifications in software have been aspirational documents. Compliance auditors read them. Senior engineers reviewed them at the start of a project. New hires occasionally referenced them during onboarding. But the day-to-day work of building software happened from a shared, imperfect, conversationally-maintained understanding of what the system was supposed to do. The specification was a reference document, not the source of truth.

This was not laziness. It was the constraint of the medium. A specification written in prose, sitting in Confluence or SharePoint or attached to an epic, cannot enforce itself. It depends on humans to read it, remember it, apply it, and update it. Between the sprint kickoff and the QA cycle, that prose document gets interpreted, re-interpreted, paraphrased in Slack threads, and slowly replaced by a shared mental model that diverges, sprint by sprint, from what anyone originally wrote down.

Spec-Driven Development, combined with AI coding agents, changes this. When the specification becomes the contract that an AI agent reads to generate code and a CI pipeline reads to validate it, the gap between what gets written and what gets built closes in a way it never could before. The specification stops being a reference document and becomes an executable artifact.

This change has consequences across the full software development lifecycle. Not as a revolutionary disruption, but as an incremental discipline upgrade that touches almost every Agile ceremony, every role's daily work, and every artifact a team produces. This article walks through those changes based on a real end-to-end build I ran. It is my honest practitioner assessment of what is genuinely different, what stays the same, and what teams adopting this approach should expect.

I run product and architecture for global manufacturing platforms at WSAudiology. SDD is interesting to me because it formalises practices I have applied in regulated environments for the last fifteen years, where specifications have always been non-negotiable. My observations come from that vantage point. Your context may differ. I will try to be honest about where the lessons travel and where they do not.

What SDD Is, Briefly

SDD is a discipline where the specification is treated as the durable artifact and code is treated as a regenerable output. A formal, structured specification defines behaviour, constraints, acceptance criteria, and edge cases. An AI agent generates code from that

specification. A CI pipeline validates that the implementation honours it. When the implementation diverges from the spec, the build fails.

Most teams adopting SDD are working toward one of three maturity levels. Spec-first means the specification is written before code is generated, but the connection between the two is manual and informal. Spec-anchored means the specification drives the AI agent's code generation and is referenced during review, but enforcement is partial. Spec-as-source means the specification is the authoritative artifact and code is entirely regenerable from it with no manual bridging. Most teams adopting SDD today aim for spec-anchored. Almost no team operates at spec-as-source in production.

The methodology is not new. Behaviour-Driven Development tried to do this in 2010. Domain-Driven Design has been encoding architectural intent in code structure for two decades. Formal specifications in regulated industries have used contracts of this kind for far longer. What is new is the executability. The AI agent makes the spec consequential in a way prose specs never were. The CI validation makes the spec enforceable in a way that no requirements document has ever been.

For deeper grounding on the methodology and tooling, the most useful pieces I found while preparing this work were Birgitta Böckeler's "Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl" on Martin Fowler's site, Den Delimarsky's "Spec-driven development with AI: Get started with a new open source toolkit" on the GitHub Blog, and Heeki Park's "Using spec-driven development with Claude Code" on Medium. All three are substantial and the article you are reading now does not attempt to replace them. The Spec Kit's own spec-driven.md in the official repository is also worth reading as the methodology's primary reference document.

The shift that matters for what follows is this: when specifications become executable, the way teams produce them, review them, refine them, and validate against them changes meaningfully. That change touches every Agile ceremony and every practitioner's daily work. The rest of this article walks through what I observed when I tested this end-to-end.

The Build

I built a Personal Bookmark Manager. URL, title, tags, notes, read or unread status, filtering, a summary dashboard. Deliberately simple, so the workflow itself would be the focus rather than the application complexity.

The stack was GitHub Spec Kit for the SDD methodology and structured slash commands, Claude Code as the AI coding agent, and GitHub for the Git repository and CI pipeline. The application itself ran on .NET 8, React, and Entity Framework Core In-Memory.

The total build, including the iteration loop and pipeline setup, took roughly six hours of focused work. Setup took about an hour. The Spec Kit phases took two to three hours. Implementation took one to two hours. The pipeline and iteration loop took the rest.

The complete build — including the specification artifacts, the `CLAUDE.md`, and the application code — is available at github.com/gubendiren/bookmark-manager. The repository also contains a step-by-step hands-on guide for anyone who wants to follow the same workflow themselves.

Walking Through the Build, Phase by Phase

The Constitution

The Spec Kit begins with a constitution: a structured document that captures the system's architectural philosophy, its core constraints, and its non-negotiable decisions before a single line of code is written.

What I fed in was straightforward: the application's purpose, its target user, the technology choices, and the broad architectural preferences I had in mind. What came back was more than I expected. The constitution surfaced implicit architectural preferences I had not explicitly articulated, encoded them as decisions the AI agent would be governed by, and produced a document that any future contributor, human or AI, would read before touching the system.

In traditional engineering practice, these decisions live in the architect's head, in tribal knowledge, or in an ADR document that gets written after the fact if it gets written at all. The constitution makes them institutional before the work starts.

Specifying Feature by Feature

I specified four features incrementally rather than all at once. The flow for each was: write the specification, review it, commit it to the repository. Each feature specification became a versioned artifact in the codebase from the moment it was written.

What this phase demanded was precision I had not fully anticipated. Writing a good behavioural specification requires distinguishing clearly between must, should, must not, and explicitly out of scope. It requires writing what the system does, not what the system is. A specification that says "the user can manage bookmarks" is not a specification. A specification that says "the system must prevent saving a URL that has already been saved and return a clear duplicate error" is.

Clarify

This was the most surprising phase.

The clarify phase runs after features are specified. It surfaces ambiguities in the specification before code is generated. Claude Code reads the spec and asks questions. Not generic questions. Specific ones about cases the spec did not address.

In my build, the questions covered things like how the system should handle tag whitespace, whether duplicate URLs across different users were in scope, how the application should respond to non-HTML content at a bookmarked URL, and whether read state should be tracked at the individual user level or at the bookmark level globally.

None of these are exotic edge cases. All of them would have become bugs, Slack threads, or last-minute scope debates during QA if they had not been surfaced here. The clarify phase automates what experienced engineers do implicitly when reading ambiguous requirements. It catches the ambiguity before it becomes a defect. This is the kind of pre-flight check that does not exist in traditional Agile workflows, and removing one whole category of mid-sprint surprise is quietly valuable.

Plan

The plan phase generates an architectural and implementation plan from the specification and the constitution. What it produced in my build was a coherent approach that reflected the architectural constraints I had encoded in the constitution, with the Repository pattern applied consistently across the data access layer.

The work that plan review represents is worth being honest about. It is not authoring. It is review. You are applying accumulated judgement about system context and constraints to a plan someone else produced. That review is still real work. You will catch things the plan missed or got wrong. But the time investment is substantially lower than producing the plan from scratch, and the quality bar for the starting point is higher than what most sprint planning sessions produce.

Tasks

The task decomposition produced roughly thirty dependency-ordered tasks from the plan. These were committed to the repository alongside the specification and the plan, giving the team a single location for every artifact that governs the build.

For anyone who has spent a sprint planning session manually decomposing work into tasks, the time saving here is immediate and substantial. The cognitive load is not the decomposition itself but the thinking that should precede it. SDD puts the thinking first. The decomposition follows automatically.

Implement Task by Task

I implemented task by task rather than asking Claude Code to generate the entire application at once. The reasons are practical. Smaller generation scopes produce more reviewable output. Errors are localised. The spec serves as a clear acceptance criterion for each task. When a task is done, the spec tells you whether it is done correctly.

In my build, I made four corrections during implementation. Three were minor, involving parameter naming or return type alignment. One was meaningful: the tag parsing diverged

from the specification's definition of valid tag format in a way that would have caused silent failures in filtering. The correction went into the spec first and the code second. That ordering matters. If you fix the code without fixing the spec, you have diverged from the contract, and downstream generations will reproduce the original error.

This is the discipline that distinguishes SDD from prompt-driven development. In prompt-driven development, when the AI gets something wrong, you fix the output. In SDD, when the AI gets something wrong, you fix the specification and regenerate. The specification is the source of truth, not the output.

PR, Pipeline, and Acceptance

Implementation proceeded through a feature branch. The PR triggered the CI pipeline, which ran a basic spec compliance check alongside the standard test suite. My compliance check was a starting point rather than a production-grade implementation. A real production CI spec compliance check needs to parse the specification structurally, map claims to test assertions, and flag gaps. What I had in the demo was directionally correct but not complete. I want to be honest about that rather than overstate what I built.

What changed about the human review step is worth noting. The CI pipeline had already pre-validated functional compliance against the specification. The human reviewer was left with the work that only humans can do well: assessing UX, evaluating trade-offs, questioning business intent. The review became faster but also more demanding. You can no longer spend your review time on things the pipeline already checked.

Iteration

Adding the Favourites feature went through the same loop. Specify, clarify, plan, decompose, implement, PR, pipeline, acceptance. The Git log told the development story in approximately twelve commits, one per SDD phase across both iterations.

This matters because the common objection to SDD is that it sounds like waterfall: you write everything upfront before you build anything. The iteration phase proves that is not what it is. The loop applies to new features, fixes, and refinements. You do not write the entire system specification before you start. You write the specification for what you are building this sprint, build it, and expand the specification in the next sprint when the scope expands.

What Changes for Agile Ceremonies

Sprint Planning

Traditional Sprint Planning spends substantial time on requirements clarification and task decomposition. Engineers ask the product manager to re-explain requirements that were

unclear in the ticket. Tasks get created in the ceremony itself. Estimates reflect uncertainty about what, exactly, is being built.

Both the clarification and the decomposition shrink dramatically when SDD has already produced the spec, plan, and tasks before the ceremony. What remains is the genuinely human work: prioritisation, capacity planning, dependency negotiation, scope decisions under real constraints. The ceremony becomes shorter but higher leverage. The conversation it contains is the conversation it should always have been, stripped of the mechanical overhead that absorbed so much of its time.

This aligns with what other writers on SDD have argued. Sprint Planning always should have been about decisions, not documentation. SDD makes that possible.

Daily Standup

The structure stays. The format stays. The content of blockers shifts.

Traditional blockers in standup are usually implementation-level: I am stuck on this code, I cannot get this API to respond correctly, the environment is broken. These are blockers that an engineer, a tech lead, or a DevOps engineer can typically unblock.

SDD blockers are usually spec-level: the specification is ambiguous about this case, the spec says one thing and the plan implies another, this edge case was not in the clarify phase and I am not sure how to handle it. These are fundamentally different kinds of blockers. The people who can unblock them need to understand the specification well enough to extend or clarify it on the spot. That fluency is worth being deliberate about when a team first adopts SDD. It does not come automatically, and the sprint stalls when nobody in the standup can answer a spec-level question.

Sprint Review and Acceptance

The CI pipeline validates spec compliance before any human review begins. By the time a PR reaches a human reviewer, the specification-level questions have already been answered by the build. The human reviewer arrives at a surface that has already been mechanically checked.

This makes the human review faster and simultaneously more demanding. Faster because the reviewer is not re-doing what the pipeline already did. More demanding because the reviewer can no longer rely on "I'll know it when I see it" as an acceptance heuristic. The reviewer is now accountable for the judgement calls that only humans can make well: whether the user experience is right, whether the trade-offs are defensible, whether the business intent is honestly served.

That is a higher bar. Most experienced practitioners will find it a better use of their time.

Sprint Retrospective

Traditional retros focus on team dynamics, process friction, and recurring blockers. SDD adds a dimension that I have not seen discussed widely in the existing writing: how well did our specifications work this sprint?

This means asking whether the constitution captured the right constraints, whether the specify phase produced specs that the clarify phase did not substantially have to correct, and whether the implementation surfaced gaps that should have been caught earlier. The spec quality retrospective turns the retro into a feedback loop for improving the specification practice itself, not just the delivery process.

This matters because the specification practice is a compounding asset. A constitution that starts incomplete improves over sprints as the team adds to it. Specs that start imprecise improve as the team gets better at writing behavioural contracts. The retro is where that improvement is named, captured, and actioned. Teams that skip this dimension of the retro will leave compounding value on the table.

Across all four ceremonies, the pattern is consistent. The structure stays. The content becomes more demanding in exactly the right places: decisions, trade-offs, and specification quality. What compresses is the mechanical overhead that was never the point of the ceremony to begin with.

What I Would Do Differently, and Where I Would Be Cautious

Do not over-spec small changes. Birgitta Böckeler makes this point in her tool comparison and I agree with it completely. The investment in a formal SDD loop pays off on features with meaningful behavioural complexity, real acceptance criteria, and a life cycle long enough to benefit from spec maintenance. For a one-line bug fix or a trivial copy change, the spec overhead is not worth it. Just fix it. The discipline of knowing when to apply SDD and when to leave it aside is part of the practice.

The SDD workflow needs a `CLAUDE.md` file to be repeatable. Without it, every new Claude Code session starts without context: it does not know it is operating inside an SDD workflow, which phases to follow, or what conventions the project has established. With a well-written `CLAUDE.md` at the root of the repository, the workflow becomes reliably automatic from the first prompt. New team members need to read that file before they touch the project. It is not optional documentation. It is the operating manual for the workflow.

Spec maintenance is real work. Adding the Favourites feature required updating the original specification to keep it consistent with the new behaviour. If you let specs drift, you lose the compounding benefit. The spec that is six sprints out of date is not enforceable. It is a legacy document. Keeping specifications current is effort, and that effort should be budgeted explicitly in sprint planning.

Be honest about which maturity level you are at. I built at spec-first, not spec-anchored, and certainly not spec-as-source. The CI compliance check I implemented is a starting point. Overclaiming the maturity of your SDD practice is its own risk: it sets expectations the practice cannot yet meet, and it makes the gaps feel like failures rather than the normal condition of a practice that is still maturing.

Closing

SDD with AI coding agents is not a paradigm shift in software development. It is a discipline upgrade. The Agile ceremonies stay. The boards, the sprints, the PRs, the retros: all of it stays. What changes is the precision required at the front of the workflow and the compounding value that precision creates downstream.

The specification, properly written and enforced, becomes an institutional asset. Every sprint that builds on it, corrects it, and extends it makes it more valuable. The constitution that was thin in sprint one is richer after sprint ten. The spec that missed edge cases in the clarify phase catches more of them after the team has run a few retros on spec quality. The practice compounds in a way that prompt-driven development does not.

For practitioners considering this approach, my recommendation is to try it on a real feature in a real project. Not a tutorial. Not a greenfield demo. A feature where the cost of getting the spec wrong is observable, and where the value of getting it right compounds across maintenance and iteration. That is where you will see whether SDD is worth the investment for your team.

For me, the answer was yes, with a qualification. I work on platforms where specifications are non-negotiable, where regulated environments have always required this kind of rigour, and where the cost of ambiguity in requirements has always been high. SDD formalises what good engineering practice has always tried to do in contexts like mine. For teams where speed-to-prototype matters more than long-term correctness, the trade-offs are different. Pick your maturity level honestly and apply the discipline where it pays off.

What seems most likely to me is that SDD becomes standard practice for software with meaningful complexity or longevity, and remains overhead for software that is prototypical or short-lived. The interesting work for teams right now is not debating whether SDD is good. It is developing the judgement to know when to apply it and when to leave it aside. That judgement, like all engineering judgement, comes from doing the work and observing what happens.

Practitioner observations are more useful when they are compared against each other. If you are exploring SDD, I would be glad to connect.

Further Reading

- Birgitta Böckeler — Understanding Spec-Driven-Development: Kiro, spec-kit, and

Tessl — martinfowler.com

- Den Delimarsky — Spec-driven development with AI: Get started with a new open source toolkit — GitHub Blog
- Heeki Park — Using spec-driven development with Claude Code — Medium
- GitHub — spec-driven.md — Official Spec Kit methodology reference
- gubendiren — bookmark-manager — Sample build and hands-on guide referenced in this article